

HealthPro · Завдання для Replit Agent

Виправлення модуля біометричної авторизації — Samsung A24 / Android 14 / One UI 6.1 / MediaTek Dimensity

Версія проєкту: v5.0.0 · Травень 2026

1. Діагноз проблеми

Поточний плагін (@capgo/capacitor-native-biometric) критично нестабільний на Samsung A24 / Android 14 / One UI 6.1 / MediaTek Dimensity.
Симптом: біометрія вискакує → зависає → не пускає в додаток (deadlock).

Три шари накладених причин:

#	Причина	Деталь
1	Samsung One UI 6.1 / Android 14	Зламаний fingerprint API після One UI 6.1 — масові репорти навіть на S23/S24
2	MediaTek Dimensity power management	Агресивний throttling: прибиває background callbacks, затримує biometric response
3	Слабка реалізація Capgo plugin	Не обробляє: onAuthenticationError, ERROR_USER_CANCELED, lifecycle activity resume/pause

2. Обране рішення

Плагін / підхід	Стабільність Samsung	Рекомендація
@capgo biometric	4 / 10	❌ Замінити
@aparajita biometric-auth	8 / 10	✅ Використовувати
Android Credential Manager	9 / 10	❌ Майбутнє (складно зараз)
Passkeys / WebAuthn	10 / 10	❌ Для офлайн-app: overkill

Висновок: Для HealthPro (офлайн-додаток, без сервера, без WebAuthn) — оптимальне рішення зараз: @aparajita/capacitor-biometric-auth + Android Keystore + fallback PIN.

3. Завдання для Replit Agent (покроково)

T1 · Замінити плагін

```
# Видалити старий плагін

npm uninstall @capgo/capacitor-native-biometric

# Встановити новий плагін
```

```
npm install @aparajita/capacitor-biometric-auth
```

```
npx cap sync
```

T2 • Створити biometric service

Файл: src/features/auth/biometric.service.ts

```
import { BiometricAuth, AndroidBiometryStrength }

from '@aparajita/capacitor-biometric-auth';

let authRunning = false;

export async function checkBiometricAvailable()

: Promise {

  try {

    const info = await BiometricAuth.checkBiometry();

    return info.isAvailable === true;

  } catch { return false; }

}

export async function biometricAuth()

: Promise {

  // Mutex: не дублювати виклики

  if (authRunning) return false;

  authRunning = true;

  // Delay: Samsung ламається без паузи

  await new Promise(r => setTimeout(r, 400));

  try {

    await Promise.race([

      BiometricAuth.authenticate({

        reason: "Вхід до HealthPro",

        allowDeviceCredential: true,

        androidBiometryStrength:

          AndroidBiometryStrength.strong,

        androidConfirmationRequired: false,

      }),

    ]),

    // Timeout 8с — Samsung може зависнути назавжди
```

```

new Promise( (_, reject) => setTimeout(

() => reject(new Error("BIOMETRIC_TIMEOUT")),

8000

))

]);

return true;

} catch (e: any) {

console.error("[Biometric] failed:", e?.message);

return false;

} finally {

authRunning = false;

}

}

```

T3 • Правильний lifecycle flow

Крок	Дія	Коментар
1	App launch	Рендерити UI — спочатку!
2	Перевірка сесії	Є збережена сесія?
3	Show unlock screen	НЕ відразу authenticate()
4	Вибір користувача	Biometric або PIN fallback
5a	Biometric success	unlock key → decrypt → open app
5b	Biometric fail/timeout	Перейти на PIN fallback
5c	PIN success	decrypt → open app

T4 • Заборонені патерни (обов'язково перевірити)

❑ НЕ МОЖНА:

- biometric-only access — якщо впаде, користувач locked out назавжди
- auto authenticate() на старті без delay
- auto authenticate() при кожному onResume / поверненні з background
- два виклики authenticate() одночасно (deadlock)
- await authenticate() без timeout (Samsung = hung promise назавжди)
- блокувати вхід якщо biometric unavailable або не зареєстрована

□ ОБОВ'ЯЗКОВО:

- timeout wrapper (8 сек) навколо кожного biometric виклику
- mutex / authRunning guard — один виклик у черзі
- fallback PIN завжди доступний (не опціонально!)
- delay 300-500ms перед authenticate() — Samsung вимагає
- AndroidBiometryStrength.strong (weak = Samsung face = нестабільний)
- allowDeviceCredential: true (якщо fingerprint впав — system PIN спрацює)
- androidConfirmationRequired: false (Samsung confirmation UI — баги)
- обробка resume/pause — HE auto-auth, лише update state

T5 · Android Keystore + encrypted session

Важливо: біометрія — це unlock механізм для ключа, HE пряма авторизація.

```
// ПРАВИЛЬНА архітектура:  
  
// Biometric success → unlock Keystore key  
  
// → decrypt local session → open app  
  
  
// НЕ:  
  
// Biometric success → grant access (без ключа)  
  
  
// Використовувати @capacitor/preferences або  
// capacitor-secure-storage-plugin для зберігання  
// зашифрованого стану сесії
```

Якщо складно реалізувати повний Keystore flow — мінімально: encrypted local session token через capacitor-secure-storage-plugin + завжди PIN fallback.

T6 · UI вимоги до екрану розблокування

Елемент	Вимога
Кнопка біометрія	Активна тільки якщо checkBiometricAvailable() = true
Кнопка PIN fallback	Завжди видима і активна
Loading state	Показувати під час biometric prompt
Timeout feedback	"Час вийшов. Використайте PIN" після 8 сек
Retry	Дозволити повторну спробу biometric
Авто-виклик	НЕ робити. Тільки за кнопкою користувача

T7 · Samsung-специфічні захисти

```
// 1. Завжди delay перед authenticate()  
  
await new Promise(r => setTimeout(r, 400));
```

```
// 2. Захист від resume auto-auth

App.addListener("appStateChange", ({ isActive }) => {

  if (isActive) {

    // Тільки оновити UI, HE authenticate()

    updateUnlockScreenState();

  }

});

// 3. Не запускати під час splash/nav transition

// Чекати поки UI повністю змонтований
```

4. Обов'язкові тест-сценарії на Samsung A24

Сценарій	Очікуваний результат
minimize → restore app	Show unlock screen, HE auto-auth
screen off → screen on	Unlock screen, biometric кнопка активна
biometric cancel (кнопка назад)	Show "спробувати знову" + PIN fallback
8 секунд без дії	"Час вийшов" → PIN fallback
Швидко 5 разів натиснути biometric	Mutex — тільки 1 prompt, без дублювання
Відключений fingerprint у налаштуваннях	Показати PIN fallback, не падати
battery optimization ON	Сервіс не вбивається, unlock працює
Після оновлення APK (OTA)	Сесія збережена, unlock доступний

5. Файлова структура нового модуля

```
src/features/auth/

biometric.service.ts ← основний сервіс (T2)

pin.service.ts ← PIN fallback логіка

session.service.ts ← encrypted session (T5)

unlock-screen.js ← UI екрану розблокування (T6)

android/app/src/main/res/xml/

network_security_config.xml ← якщо потрібно
```

6. Залежності npm

```
# Видалити

npm uninstall @capgo/capacitor-native-biometric
```

```
# Встановити

npm install @aparajita/capacitor-biometric-auth

# Опціонально для secure storage:

npm install capacitor-secure-storage-plugin

npx cap sync
```

7. Definition of Done (чеклист)

☐ / ☐	Критерій
☐	@capgo biometric видалений
☐	@aparajita biometric встановлений
☐	Timeout 8 сек реалізований для кожного виклику
☐	Mutex authRunning guard реалізований
☐	Delay 400ms перед authenticate()
☐	allowDeviceCredential: true
☐	androidBiometryStrength: strong
☐	androidConfirmationRequired: false
☐	PIN fallback завжди доступний
☐	Biometric HE є обов'язковою для входу
☐	Авто-auth при resume — відсутня
☐	Авто-auth при старті — відсутня
☐	Всі 8 Samsung тест-сценаріїв пройдені
☐	npx cap sync виконаний
☐	APK зібраний та протестований на Samsung A24

Важливо: Biometric API на Android HE є гарантованою фічею. Google прямо вказує в security model: biometric = unlock mechanism, HE authentication. Тому fallback PIN — обов'язковий, не опціональний. Джерело аналізу: ChatGPT розмова про проблеми Samsung A24 + Capacitor biometric (травень 2026). Плагін @aparajita = найкращий вибір зараз для офлайн-апп без backend/WebAuthn.